Convolution & the Receptive Field, Worked Out by Hand

Sliding dot products, output sizes, and how the field grows

A complete numerical walk-through of one 3×3 kernel sliding over one 5×5 image. We compute the entire feature map by hand — every sliding dot product written out — then derive the output-size formula and tabulate it, and finish by tracking how far back into the input a single deep neuron can "see" as we stack layers. Every number below was produced by the NumPy/PyTorch reference program in Appendix A and cross-checked against `torch.nn.functional.conv2d`, so the arithmetic is exact and self-consistent.

What this document does. It fixes a single grayscale image with a clean vertical edge — the left three columns dark (0), the right two columns bright (1) — and a vertical Sobel kernel, then computes the 3×3 valid feature map position by position. It writes out the explicit weighted sum at a flat corner (response 0) and at the edge (response -4), states the output-size formula and applies it to seven (K, S, P) settings, and traces the receptive field through three stacked convs.

How to read this. Each step isolates one mechanism and shows it on the cleanest possible numbers, with a *Mini-example* box stripping it to its core. The running edge image is small enough to verify with a pencil; the formulas are exactly the ones `conv2d` uses internally, just written out.

Concepts covered, in order: the convolution (really cross-correlation) operation as a sliding dot product · weight sharing & locality · the valid feature map computed elementwise · why edge patches respond strongly · the output-size formula $\lfloor (W - K + 2P)/S \rfloor + 1$ · `valid` vs `same` padding · the receptive field and its recurrence · how stride-2 layers grow the field faster.

1 The setup: one image, one kernel, small enough to compute by hand

A convolution slides a small **kernel** (filter) across an input, and at each stop computes a dot product between the kernel and the patch of pixels underneath it. The output is a **feature map** that lights up wherever the kernel's pattern appears. To see every number we shrink to a 5×5 single-channel image and a 3×3 kernel.

The image. A clean *vertical edge*: the left three columns are dark and the right two are bright. Each pixel is its own value, so nothing is hidden:

$$X = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix} \qquad \text{(edge sits between columns 2 and 3).}$$

The kernel. The vertical Sobel filter, which responds to left→right brightness changes:

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}.$$

Its left column is $+$, its right column is $-$, and the middle is 0: it measures "how much brighter is the left of this patch than the right." A flat patch gives 0; a left-bright/right-dark patch gives a large positive number; a left-dark/right-bright patch (our edge) gives a large negative number.

Quantity	Symbol	Value here
Input height / width	H, W	5, 5
Kernel height / width	K	3×3
Stride	S	1
Padding	P	0 (`valid`)
Output height / width	H', W'	3, 3
Channels (in, out)	—	1, 1

Intuition

A convolution is two ideas at once: **locality** — each output looks at only a tiny 3×3 neighbourhood, not the whole image — and **weight sharing** — the *same* nine kernel numbers are reused at every position. One small filter scans the entire image. That is why a conv layer has so few parameters (9 here, regardless of image size) and why it detects a pattern *wherever* it occurs: the edge detector fires the same way on the left edge of one object as on the left edge of another.

Watch out

What deep-learning libraries call "convolution" is technically *cross-correlation*: the kernel is *not* flipped before the dot product. True mathematical convolution flips the kernel both ways. The two differ only by a kernel reflection, and since the network *learns* the kernel, the distinction is

irrelevant in practice — but it matters if you compare against a signal-processing reference. Everything below uses the cross-correlation convention, matching `torch.nn.functional.conv2d`.

2 Step 1 — One output pixel: a single sliding dot product

Place the kernel's top-left corner at input position (i, j). The output value $Y[i, j]$ is the sum of the 9 elementwise products of the kernel with the 3×3 patch $X[i{:}i{+}3,\ j{:}j{+}3]$:

$$\boxed{Y[i,j] \;=\; \sum_{u=0}^{2}\sum_{v=0}^{2} X[i+u,\ j+v] \cdot K[u,v]}$$

With $S = 1$ and $P = 0$ the corner (i, j) ranges over $i, j \in \{0, 1, 2\}$, giving a 3×3 output.

Position $(0, 0)$ — **the flat corner.** The patch is entirely in the dark region:

$$X[0{:}3,\ 0{:}3] = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \text{patch} \odot K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad Y[0,0] = 0.$$

A uniform patch always gives 0: the kernel's weights sum to 0, so a constant region cancels exactly. *No edge ⇒ no response.*

Position $(0, 1)$ — **straddling the edge.** Shift one column right; now the patch's right column lands on the bright side:

$$X[0{:}3,\ 1{:}4] = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}, \quad \text{patch} \odot K = \begin{bmatrix} 0 & 0 & -1 \\ 0 & 0 & -2 \\ 0 & 0 & -1 \end{bmatrix}, \quad Y[0,1] = -4.$$

Term by term: only the right column is nonzero, contributing $1{\cdot}(-1) + 1{\cdot}(-2) + 1{\cdot}(-1) = -4$. The sign is negative because the *right* of the patch is brighter than the left — exactly the transition the vertical Sobel measures, and the largest-magnitude response in the whole map.

Mini-example — this operation in isolation

Why the magnitude is 4 and not, say, 1: the Sobel kernel weights its middle row by 2 (the rows are $1, 2, 1$). Crossing the edge, the three right-column pixels are all 1, so the response is $-(1 + 2 + 1) = -4$. The "$1, 2, 1$" weighting is a small built-in blur along the edge direction, making the detector less twitchy to single-pixel noise than a plain $[1, 0, -1]$ row would be.

3 Step 2 — The full 3×3 feature map

Sliding the corner over all nine positions $i, j \in \{0, 1, 2\}$ and evaluating the boxed sum each time gives the complete valid output:

$$Y = \begin{bmatrix} 0 & -4 & -4 \\ 0 & -4 & -4 \\ 0 & -4 & -4 \end{bmatrix}.$$

Read it column by column. Column $j = 0$ of the output corresponds to patches sitting entirely in the dark-left region (corner columns $0, 1, 2$), so every entry is 0. Columns $j = 1, 2$ correspond to patches whose right edge has crossed into the bright region, so every entry is -4. The feature map has isolated the one vertical edge in the image into a band of strong (negative) response, with flat regions silenced to 0.

Intuition

Notice every *row* of Y is identical: $[\,0, -4, -4\,]$. That is **translation equivariance** made visible. The image's edge is the same at every height, so the detector responds the same at every height. Slide the pattern and the response slides with it; this is the property that lets one trained kernel recognise a feature anywhere in an image without relearning it per location.

When this is the right choice

Convolutions are the right tool when your data has **local grid structure** — where *nearby* samples are meaningfully related and the same local pattern can appear anywhere. Images (2-D grid of pixels), spectrograms (time × frequency), and many time-series (1-D grid where local shape matters) all qualify. The locality and weight-sharing that make convolution cheap are exactly wrong for data with no spatial notion — e.g. tabular rows where column order is arbitrary — there a fully-connected layer is the honest choice.

4 Step 3 — The output-size formula

The output is smaller than the input because the kernel cannot hang off the edge: its corner can only sit where a full $K \times K$ patch fits. With stride S (the step between corners) and padding P (rings of zeros added around the input), the number of valid positions along one axis is

$$\boxed{\text{out} \;=\; \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1}$$

For our case $W = 5,\ K = 3,\ P = 0,\ S = 1$:

$$\text{out} = \left\lfloor \frac{5 - 3 + 0}{1} \right\rfloor + 1 = \lfloor 2 \rfloor + 1 = 3,$$

matching the 3×3 map we computed. The "+1" counts the starting position; the $\lfloor \cdot \rfloor$ discards a final partial step the kernel cannot complete.

W	K	S	P	out	note
5	3	1	0	3	our valid case
5	3	1	1	5	`same`: pad 1 preserves size
5	5	1	0	1	kernel fills the image: one position
5	3	2	0	2	stride 2 roughly halves
5	3	2	1	3	stride 2 with pad
32	3	1	1	32	`same` on a real feature map
32	3	2	1	16	stride-2 downsample, exact halving

The two rules of thumb worth memorising sit in this table: a 3×3 kernel with $P = 1,\ S = 1$ keeps the spatial size unchanged (`same`), and 3×3 with $P = 1,\ S = 2$ halves it. Both are the workhorses of modern CNN design.

Watch out

The two classic off-by-one traps. **(1) `valid` vs `same`.** With no padding (`valid`) every conv shrinks the map by $K - 1$; stack ten 3×3 convs and you have lost 20 pixels per axis. To preserve size (`same`) use $P = (K-1)/2$, i.e. $P = 1$ for $K = 3$, $P = 2$ for $K = 5$. **(2) The floor.** When $S > 1$ the $\lfloor \cdot \rfloor$ silently drops pixels: $W = 5, K = 3, S = 2, P = 0$ gives $\lfloor 2/2 \rfloor + 1 = 2$, *not* 3 — the last column is never visited by a kernel corner. Always run the formula; do not eyeball it.

Mini-example — this operation in isolation

Padding to keep size, by hand. Wrap our 5×5 image in one ring of zeros ($P = 1$), making it 7×7, then slide the 3×3 kernel with $S = 1$: now $\lfloor (5 - 3 + 2)/1 \rfloor + 1 = 5$, a 5×5 output — same as the input. The price is that border outputs are computed against invented zeros, so the leftmost / rightmost columns of a Sobel map pick up an artificial edge where the real image meets the pad.

5 Step 4 — The receptive field: how far one neuron sees

A single output pixel of one 3×3 conv depends on a 3×3 patch of its input. Stack a second 3×3 conv and each of *its* outputs depends on a 3×3 patch of the *first* map — but each of those depends on a 3×3 input patch, so the second layer's pixel traces back to a 5×5 region of the original image. This back-traced region is the **receptive field** (RF): the area of the input that can influence one output neuron.

The RF grows by a recurrence. Let j_ℓ be the *jump* (the product of all strides *before* layer ℓ, i.e. how many input pixels apart adjacent outputs of layer $\ell - 1$ are). Then

$$\boxed{\mathrm{RF}_\ell \;=\; \mathrm{RF}_{\ell-1} + (k_\ell - 1) \cdot j_\ell, \qquad j_\ell = \prod_{m<\ell} s_m, \qquad \mathrm{RF}_0 = 1.}$$

Each new layer adds $(k_\ell - 1)$ steps, and each step spans j_ℓ input pixels.

Three 3×3 stride-1 convs. Every stride is 1, so the jump stays $j_\ell = 1$ throughout and each layer simply adds $k - 1 = 2$:

$$\begin{aligned} \mathrm{RF}_1 &= 1 + (3 - 1) \cdot 1 = 3, \\ \mathrm{RF}_2 &= 3 + (3 - 1) \cdot 1 = 5, \\ \mathrm{RF}_3 &= 5 + (3 - 1) \cdot 1 = 7. \end{aligned}$$

So the field grows $3 \to 5 \to 7$: three 3×3 layers see as much input as a single 7×7 kernel — but with $3 \times (3{\cdot}3) = 27$ weights instead of $7{\cdot}7 = 49$, and two extra nonlinearities. This is precisely why modern CNNs stack small kernels rather than use one big one.

Intuition

The jump j_ℓ is the key to why stride matters. With all stride-1 layers, j stays 1 and the field grows *linearly* (+2 per layer). The moment a stride-2 layer appears, every *later* layer's steps each cover 2 input pixels, so subsequent additions double in reach. Stride doesn't just shrink the feature map — it accelerates how fast deeper neurons gain context.

Inserting a stride-2 layer. Replace the middle conv with a stride-2 conv: layers $(k, s) = (3,1),(3,2),(3,1)$. The jump is 1 for layers 1 and 2 (no stride yet before them), then becomes $1 \cdot 2 = 2$ for layer 3:

$$\begin{aligned} \mathrm{RF}_1 &= 1 + (3-1)\cdot 1 = 3, \quad j_1 = 1, \\ \mathrm{RF}_2 &= 3 + (3-1)\cdot 1 = 5, \quad j_2 = 1, \\ \mathrm{RF}_3 &= 5 + (3-1)\cdot 2 = 9, \quad j_3 = 1\cdot 2 = 2. \end{aligned}$$

The same three layers now reach 9×9 instead of 7×7: the single stride-2 step widened layer 3's contribution from +2 to +4. The course's 5-layer all-stride-1 3×3 network continues this pattern to RF = 11.

Layer ℓ	config (k_ℓ, s_ℓ)	jump j_ℓ	RF_ℓ (all stride-1)	RF_ℓ (layer 2 stride-2)
1	$(3,1)$	1	3	3
2	$(3,\cdot)$	1	5	5
3	$(3,1)$	1 / 2	7	9

6 Convolution, at a glance

#	Quantity	For our example	Computed as
1	one output	$Y[i,j]$	$\sum_{u,v} X[i{+}u, j{+}v]\, K[u,v]$
2	flat-patch response	0	uniform patch, weights sum to 0
3	edge response	-4	$-(1{+}2{+}1)$, right column bright
4	full feature map	$\begin{bmatrix} 0 & -4 & -4 \end{bmatrix}$ per row	slide corner over 3×3 grid
5	output size	3×3	$\lfloor (5-3+0)/1 \rfloor + 1$
6	`same` padding	$P = 1 \Rightarrow 5 \times 5$	$P = (K-1)/2$
7	RF, 3 stride-1 convs	7	$1+2+2+2$
8	RF, with stride-2 mid	9	$1+2+2+(2{\cdot}2)$

These eight lines are the whole topic: a sliding weighted sum, a size bookkeeping rule, and a recurrence for how much context a deep neuron commands.

7 Equation cheat-sheet

Convolution (cross-corr.)	$Y[i,j] = \sum_{u,v} X[i{+}u,\ j{+}v]\, K[u,v]$
Output size (per axis)	$\text{out} = \lfloor (W - K + 2P)/S \rfloor + 1$
`same` padding ($S{=}1$)	$P = (K-1)/2 \quad$ (so out $= W$)
Parameters (1 in, 1 out ch.)	$K \times K$ weights ($+1$ bias), reused at all positions
Receptive field recurrence	$\text{RF}_\ell = \text{RF}_{\ell-1} + (k_\ell - 1)\, j_\ell$
Jump (effective stride)	$j_\ell = \prod_{m<\ell} s_m, \qquad \text{RF}_0 = 1$
Stacked 3×3 stride-1	$\text{RF} = 2L + 1 \quad (L$ layers: $3, 5, 7, 9, \dots)$

Appendix A Reference implementation

Every number above is reproducible from the following program. The hand-rolled NumPy convolution is checked against `torch.nn.functional.conv2d`; they agree exactly.

```
import numpy as np
import torch
import torch.nn.functional as F

# --- input: 5x5 vertical edge (left 3 cols dark, right 2 cols bright) ---
X = np.zeros((5, 5))
X[:, 3:] = 1.0

# --- vertical Sobel kernel ---
K = np.array([[1, 0, -1],
              [2, 0, -2],
              [1, 0, -1]], dtype=float)

# --- VALID cross-correlation, stride 1, no padding ---
def conv2d_valid(X, K):
    kh, kw = K.shape
    H, W = X.shape
    Y = np.zeros((H - kh + 1, W - kw + 1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            Y[i, j] = np.sum(X[i:i+kh, j:j+kw] * K)
    return Y

Y = conv2d_valid(X, K)
print(Y)                      # [[0 -4 -4],[0 -4 -4],[0 -4 -4]]

# explicit weighted sums
print(np.sum(X[0:3, 0:3] * K))   # 0   (flat corner)
print(np.sum(X[0:3, 1:4] * K))   # -4  (straddles the edge)

# --- match torch ---
Xt = torch.tensor(X).reshape(1, 1, 5, 5)
Kt = torch.tensor(K).reshape(1, 1, 3, 3)
Yt = F.conv2d(Xt, Kt, stride=1, padding=0)
print(np.max(np.abs(Yt.reshape(3, 3).numpy() - Y)))   # 0.0  (exact match)
print(F.conv2d(Xt, Kt, padding=1).shape)              # (1,1,5,5)  'same'

# --- output-size formula ---
def out_size(W, K, P, S):
    return (W - K + 2 * P) // S + 1

for (k, s, p) in [(3,1,0),(3,1,1),(5,1,0),(3,2,0),(3,2,1)]:
    print(k, s, p, "->", out_size(5, k, p, s))   # 3,5,1,2,3

# --- receptive field growth ---
def receptive_field(layers):       # layers = list of (k, s)
    rf, jump = 1, 1
    for k, s in layers:
        rf = rf + (k - 1) * jump
        jump *= s
        print("RF =", rf, " jump =", jump)
    return rf
```

```
print("three 3x3 stride-1:")
receptive_field([(3,1),(3,1),(3,1)])     # 3, 5, 7
print("insert stride-2 in the middle:")
receptive_field([(3,1),(3,2),(3,1)])     # 3, 5, 9
```

— end of document —